Distributed Systems

Paxy Report
Team M3 #1.01

Autumn 2026

Code: CPDS

Inderjot Singh (inderjot.singh@estudiantat.upc.edu)

Fawzi Daoud (fawzi.daoud@estudiantat.upc.edu)

Ida Kallevik Tollaksen(ida.kallevik.tollaksen@estudiantat.upc.edu)

UPC

October 2026

Upload your report in PDF format.
Use this LaTeX template to format the report.
Validate your basic Paxy code through the review questionnaires at ATENEA.
Submit a compressed file (.tar.gz) containing ALL your source code files.

1 Introduction

In this seminar we worked with Paxy, an Erlang implementation of the Paxos algorithm. Paxos addresses a central problems in distributed systems, consensus. Meaning how a group of processes can agree on a single value even when messages are delayed or lost and some processes crash. After completing the acceptor and proposer files, we looked at how the algorithm behaves under message delays, lost and missing `sorry` messages, and different numbers of acceptors and proposers. We also ran it across several Erlang nodes, made the acceptors fault tolerant, and improved the proposer using `sorry` messages.

2 Experiments and Questions

2.1 Message delays

Experiment. *Try introducing different delays in the* `promise` *messages,* `vote` *messages, or both sent by the acceptor. Provide figures or tables with experimental results comparing the time and the number of rounds needed to get consensus.*

Question. *Discuss the impact of message delays, and how it depends on the value of the timeout at the proposer.*

The acceptor sends its `promise` and/or `vote` replies with `timer:send_after(T, Pid, Msg)`, where `T = rand:uniform(D)`, meaning each reply is delayed by a uniform random time in $[1, D]$ ms. Each configuration was run 10 times (5 acceptors, 3 proposers), with two start patterns: *staggered* (sleeps 1000/2000/3000 ms) and *simultaneous* (100/100/100 ms). We report the average number of rounds per proposer and the average time until all proposers decided Even without delays the average is above one round (1.33 staggered, about 1.85 simultaneous).

		max delay D (ms)				
start	delayed	0	500	1000	2000	4000
staggered	promise	1.33 / 4011	1.33 / 4318	1.33 / 4644	1.43 / 5541	2.23 / 11015
	vote	1.33 / 4008	1.33 / 4274	1.33 / 4545	1.33 / 4819	1.57 / 7056
	both	1.33 / 4010	1.33 / 4601	1.33 / 4973	1.47 / 6605	3.23 / 17088
simultaneous	promise	1.87 / 3318	1.83 / 3610	1.83 / 4291	1.90 / 5960	2.50 / 9538
	vote	1.93 / 3721	1.80 / 3174	1.90 / 3995	1.90 / 4570	1.90 / 5903
	both	1.87 / 3319	1.87 / 4169	1.80 / 4424	1.87 / 6387	2.80 / 13418

Table 1: Avg. rounds per proposer / avg. total time (ms), $T = 2000$ ms. All runs terminated and agreed on a single value.

	proposer timeout T (ms)				
D (ms)	250	500	1000	2000	4000
500	2.57 / 1565	1.83 / 1859	1.87 / 2502	1.83 / 3881	1.80 / 6581
2000	–	6.83 / 11316*	2.87 / 6762	1.80 / 6641	1.87 / 9245

Table 2: Avg. rounds / avg. total time (ms) when both promises and votes are delayed, simultaneous start. With $D = 2000$, $T = 250$ no run terminated; *only 4 of 10 runs terminated (average over those). All other configurations terminated 10/10.

Does the algorithm still terminate? Yes in all runs of Table 1, even with delays twice the timeout, and no run ever produced two different decisions: delays affect only liveness, never safety. Termination is however not guaranteed: with $D = 2000$ and a short timeout.

Does it require more rounds? Only when the delay is comparable to the timeout. For $D \leq 1000$ the rounds match the no-delay baseline and the delay only adds latency. At $D = 4000$ the rounds grow, and delaying promises is clearly worse than delaying votes.

How does the impact depend on the timeout? It depends on how D compares to T. With $T \ll D$ proposers abort ballots that would have succeeded, leading to many rounds or no progress at all; with $T \gg D$ the rounds return to the baseline but every ballot that really failed costs the full timeout, so the total time grows with T. The best results were obtained for T between about $D/2$ and D.

2.2 `sorry` messages

Experiment. *Avoid sending* `sorry` *messages by commenting the corresponding sentences in the acceptor. Compare the time and the number of rounds needed to get consensus with and without* `sorry` *messages.*

Question. *Discuss the impact of dropping* `sorry` *messages, particularly, whether you could still come to an agreement.*

The two lines that send `sorry` messages in the acceptor were commented out, so a refused `prepare` or `accept` is silently ignored. We compared both versions in four settings (see table below).

		with `sorry`		without `sorry`	
start	max delay (ms)	rounds	time (ms)	rounds	time (ms)
staggered	0	1.33	4009	1.33	4009
	2000	1.45	6825	1.48	6890
simultaneous	0	1.83	3116	1.83	3115
	2000	1.80	6115	1.77	5935

Table 3: Average rounds per proposer and average total time, 20 runs per row. All runs terminated with 0 disagreements.

Agreement is still reached in every run, and the number of rounds and the time are almost the same with and without `sorry` messages. This is expected for two reasons. First, `sorry` messages are not part of the safety argument of Paxos: an acceptor that refuses may simply stay silent, and a proposer that does not collect a majority of promises or votes times out and retries in a higher round. Second, the given proposer discards them: the `sorry` clauses in `collect/4` and `vote/2` just keep waiting with the same counter. A proposer that is refused by all acceptors therefore still waits the full timeout. So dropping `sorry` messages only reduces the message traffic.

2.3 Lost messages

Experiment. *Try randomly dropping* `promise` *messages,* `vote` *messages, or both in the acceptor. Provide figures or tables with experimental results comparing the time and the number of rounds needed to get consensus.*

Question. *Discuss the impact of dropping messages, and indicate the percentage of messages we can drop until consensus is no longer possible.*

The acceptor drops each `promise` and/or `vote` reply with probability p, as if the network lost the reply; `prepare`, `accept` and `sorry` messages are never dropped. Proposers start staggered (1000/2000/3000 ms), $T = 2000$ ms, 10 runs per configuration. A run was stopped when no proposer decided for 60 s.

	drop probability p					
dropped	0%	10%	20%	30%	40%	50%
promise	1.33 / 4155	1.37 / 4212	1.40 / 4311	1.50 / 4712	1.90 / 5920	2.43 / 8172
vote	1.33 / 4009	1.33 / 4011	1.33 / 4009	1.67 / 5017	1.90 / 6029	3.03 / 9865
both	1.33 / 4009	1.33 / 4008	1.43 / 4310	1.93 / 6123	3.03 / 11021	3.63 / 11487

Table 4: Avg. rounds per proposer / avg. total time (ms). All 180 runs terminated with 0 disagreements.

p (both)	P(ballot succeeds)	runs terminated	proposers decided	avg. time (ms)
50%	25%	10/10	30/30	18948
60%	10%	3/10	20/30	29979*
70%	2.7%	0/10	7/30	–
80%	0.3%	0/10	0/30	–

Table 5: Dropping both promises and votes at high rates. P(ballot succeeds) is the probability that at least 3 of 5 replies survive in both phases. *Average over the 3 terminated runs. 0 disagreements in all runs.

Impact of dropping messages. Since a proposer only needs 3 of the 5 replies in each phase, losing a few messages does not matter much. Up to 20% loss the number of rounds stays close to the baseline of 1.33. From 30% onwards it grows quickly, and so does the time, because every failed ballot costs a full timeout. The results also vary a lot at high loss: the 50% configuration gave 3.63 rounds in one series and 5.20 in the other, with single runs taking between

10 and 54 s. Agreement was never broken. When a vote is lost, the acceptor has still stored it, so the next round learns the chosen value in phase 1 and proposes it again.

How many messages can we drop? With 50% of promises and votes lost, all runs still reached consensus; at 60% only 3 of 10 did, at 70% none, and at 80% no proposer decided at all. With our settings we can therefore drop about 50% of promise and vote messages. In theory, consensus remains possible for any $p < 100\%$, since every ballot has a non-zero probability of success, but the expected number of ballots grows from 4 at 50% to about 300 at 80%. It becomes impossible only when the replies of more than 2 acceptors are always lost, which is the same as 3 of the 5 acceptors crash.

2.4 Number of *acceptors* and *proposers*

Experiment. *Try increasing the number of acceptors and proposers. Provide figures or tables with experimental results comparing the time and the number of rounds needed to get consensus.*

Question. *Discuss the impact of having more acceptors while keeping the same number of proposers and having more proposers while keeping the same number of acceptors. Refer also to the fault tolerance and the message traffic.*

Run	Acceptors	Proposers	Max round	Time (ms)	Decided value
Increasing acceptors (proposers fixed at 3)					
1	7	3	3	4024	Blue
2	9	3	3	4017	Blue
3	11	3	3	4019	Blue
Increasing proposers (acceptors fixed at 5)					
4	5	5	3	4028	Blue
5	5	7	4	6065	Cyan
6	5	9	4	6021	Cyan

Table 6: Highest round reached and total time until all proposers decided, one run per configuration. All runs terminated with 0 disagreements.

Increasing acceptors As runs 1–3 in Table 6 show, adding more acceptors doesn't really change anything in terms of speed. With 7, 9, and 11 acceptors the run took about 4 seconds and reached round 3. This makes sense because rounds are lost when proposers compete with each other, and the number of acceptors didn't affect that. What increasing the number of acceptors does affect is the fault tolerance: since only a majority is needed, 7 acceptors can survive 3 crashes, 9 can handle 4, and 11 can handle 5. The downside is more message traffic, since every proposer has to talk to every acceptor each round.

Increasing proposers Runs 4–6 in Table 6 show something a bit different. Going from 5 to 7 proposers increased the time from about 4 to 6 seconds, and

the highest round from 3 to 4. With more proposers, they keep interrupting each other's ballots, and each failed round costs the 2-second timeout before trying again. 9 proposers took about as long as 7, but since we only did one run of each and the results vary between runs, we didn't read too much into that. More proposers also means more messages, since each one sends messages to all acceptors in every round. They don't make the system tolerate more acceptor crashes, but the system can still reach a decision if one proposer fails. In all runs every proposer agreed on the same value, so adding proposers made consensus slower, but not unsafe.

2.5 Distributed execution

Code. *Discuss the changes needed in the* `paxy` *module to create the proposers in a remote Erlang instance (named* `paxy-pro`*) and to ensure that they can connect correctly to the acceptors, which must be created in a different remote Erlang instance (named* `paxy-acc`*).*

Experiment. *Record a video providing empirical evidence of the distributed execution working properly.*

Code. First we added the names of the two remote nodes, `paxy-acc` and `paxy-pro`. Then, instead of starting the acceptors locally, we start them on `paxy-acc` with `spawn(?ACC_NODE, ...)`, so they get registered with their names on that node. When they are all ok, a message is sent back to say they are ready. We wait for that message before starting the proposers on `paxy-pro`, so they never try to talk to acceptors that dose not exist yet.

The proposers also need to know where the acceptors are. So instead of just `homer`, they get `{homer, 'paxy-acc@Mac'}`. Erlang can send messages to a name on another node like this, so the proposer code did not need any changes. We also updated `stop()` so it sends the stop message to the acceptors on `paxy-acc`. To run everything, each node is started in its own terminal with `erl -sname <name> -setcookie paxy`, from the same folder.

Experiment. In the video we start three nodes: `paxy-acc` for the acceptors, `paxy-pro` for the proposers, and `paxy`, where the GUI runs and where we call `start`. After calling `paxy:start([1000,2000,3000])`, all three proposers agree on the same colour(Red), after 4 seconds. This shows that the remote nodes are used. `paxy` waits for the `acceptors_ready` message from `paxy-acc` before starting the proposers, and the proposers can only reach the acceptors through `{Name, 'paxy-acc@Mac'}`. Meaning the run cant finish if any of the remote nodes was missing.

Video. See the video of the distributed execution here. It can also be found in the zip-file under task2.5.mp4.

2.6 Fault tolerance

Code. *Discuss the changes needed in the acceptor to make it fault tolerant.*

Experiment. *Record a video demonstrating that a crashed acceptor can recover successfully with exactly the same state it had before failing.*

Code. An acceptor only keeps its promise and its last vote in memory, so if it crashes it forgets them. Then it could accept a proposal that it had promised to ignore, and two different values could be chosen. To avoid this we use the `pers` module to save the state of the acceptor (the promise, the vote, the value and the GUI panel id) on disk. Every time the acceptor gives a promise or votes, it saves the new state first and sends the reply after. This way, if it crashes after saving, the proposer just gets no answer and tries again. If it replied first, the proposer could count a promise that the acceptor has already forgotten.

When an acceptor starts, it reads the saved state instead of using the empty values, and it updates the GUI with it. After a crash the acceptor is started with `na` as the panel, so the panel id also comes from the saved state. The state is only saved when it changes, so a `sorry` or a restart does not write anything. When the acceptor stops normally it closes and deletes the file, so the next run starts clean. We also changed `send/2` in the proposer so it first checks that a local acceptor is registered, to avoid an error when it is down.

Experiment. We started Paxy with `paxy:start([1000,2000,3000])` and waited until all the proposers decided on red. The last state of Homer before the crash was promised {`1,bender`}, voted {`1,bender`} and colour {`255,0,0`}. Then we crashed it with `paxy:crash(homer)`, which returned `true`, and `whereis(homer)` showed that Homer was running again. We read the saved state with `pers:read(homer)` and it gave {{`1,bender`},{`1,bender`},{`255,0,0`},`Panel`}, so the promise, the vote and the value are the same as before the crash. At the end we closed the table and stopped Paxy with `paxy:stop()`.

Video. See the video of the crash and recovery here.

2.7 Improvement based on `sorry` messages

Code. In the original implementation, the `collect` and `vote` procedures wait for responses from a quorum of acceptors. If too many acceptors reject a ballot, the proposer may continue waiting until the timeout expires, even though the current ballot can no longer succeed.

To improve this behaviour, we modified both procedures to count `sorry` messages. With five acceptors, a quorum requires three acceptors, so up to two rejections can be tolerated. If the number of counted rejections exceeds this limit, the current ballot is aborted immediately. The proposer then waits for a short random backoff period, increases its ballot number, and retries. This mechanism is implemented in both the prepare and accept phases.

Experiment. We compared the original and improved implementations using five acceptors and three proposers started simultaneously with `paxy:start([0,0,0])`. Each version was tested three times. Table 7 shows the total execution time for each run.

Table 7: Consensus execution time with and without the improvement.

Test	Original (ms)	Improved (ms)
1	379	40
2	16	19
3	21	21
Average	138.7	26.7

All six runs completed successfully, and all three proposers agreed on the same value, `{0,0,255}`. The average execution time was approximately 138.7 ms for the original implementation and 26.7 ms for the improved implementation, an observed reduction of approximately 80.8%.

During an additional verification run of the improved implementation, the proposer received prepare-phase `sorry` messages with counted rejection values of two and three. Since the maximum tolerated count was two, the third counted rejection triggered the abort-and-retry path. The proposer subsequently advanced to a higher ballot, and consensus completed successfully. This verifies the prepare-phase early-abort behaviour; the accept-phase rejection branch was not directly observed in this run.

However, the improved implementation was not faster in every test: Test 2 took 19 ms compared with 16 ms in the original version, while Test 3 took 21 ms for both versions. The original results also varied considerably, particularly because Test 1 took 379 ms. Since concurrent execution is affected by process scheduling and acceptor state, these measurements are preliminary and do not prove that the early-abort mechanism alone caused the reduction. Further controlled experiments would be needed to confirm its performance benefit.

3 Personal opinion

Provide your personal opinion of the seminar, indicating whether it should be included in next year's course or not.

Ida Kallevik Tollaksen I thought it was a useful and quite fun seminar. It was a good learning experience to finish the code, but it was helpful to have some structure to start with, which I liked. We divided the tasks between us, so I did not work on all of the tasks, but the questions I worked on were fine. I would perhaps want a bit more clarification of how much is expected to be written. I also did Task 2.5, where we were supposed to record a video. I thought that was fine, but again, I would have liked more clarification on what I needed to include. Overall, I think the seminar should be included in next year's course.